MoT: Modularization-of-Thought Prompting for Effective Code Generation

RUWEI PAN, Chongqing University, China
HONGYU ZHANG, Chongqing University, China

Large Language Models are transforming software development by automatically generating code. Current prompting techniques such as Chain-of-Thought (CoT) suggest tasks step by step and the reasoning process follows a linear structure, which hampers the understanding of complex programming problems, particularly those requiring hierarchical solutions. Inspired by the principle of modularization in software development, in this work, we propose a novel prompting technique called MoT (Modularization of Thought), to enhance the code generation performance of LLMs. First, MoT exploits modularization principles to decompose complex programming problems into smaller, independent reasoning steps, enabling a more structured and interpretable problem-solving process. This hierarchical structure improves the LLMs' ability to comprehend complex programming problems. Then, it structures the reasoning process using an MLR Graph (Multi-Level Reasoning Graph), which hierarchically organizes reasoning steps. This approach enhances modular understanding and ensures better alignment between reasoning steps and the generated code, significantly improving code generation performance. Our experiments on two advanced LLMs (GPT-4o-mini and DeepSeek-R1), comparing MoT to six baseline prompting techniques across eight benchmarks, demonstrate that MoT significantly outperforms existing baselines (e.g., CoT and SCoT), achieving Pass@1 scores ranging from 58.1% to 95.1%.

CCS Concepts: • **Software and its engineering → Automatic programming**.

Additional Key Words and Phrases: Code Generation, Modularization, Large Language Models

ACM Reference Format:
Ruwei Pan and Hongyu Zhang. 2026. MoT: Modularization-of-Thought Prompting for Effective Code Generation. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA082 (October 2026), 22 pages. https://doi.org/10.1145/3832173

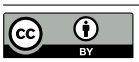
arXiv:2503.12483v2 [cs.SE] 17 Jul 2026

1 Introduction

Large Language Models (LLMs) are transforming the field of software development [1, 2, 4, 7, 19]. In recent years, an increasing number of LLMs have been developed to assist programmers in writing code, such as GPT-4 [34] and DeepSeek [10].

Various prompting techniques have also been introduced to enhance LLM-based code generation, as shown in Figure 1. For example, Chain-of-Thought (CoT) prompting [44] is one of the most widely used prompting techniques, which employs intermediate steps to facilitate step-by-step reasoning in LLMs. CoT has been shown to significantly improve LLMs' problem-solving capabilities without requiring modifications to the model itself. To further enhance the quality of code generation, Li et al. [22] proposed Structured Chain-of-Thought (SCoT), which builds upon CoT prompting by leveraging program structures (i.e., sequence, branch, and loop structures) to generate intermediate reasoning steps. Jiang et al. [18] introduced Self-Planning prompting, which enables LLMs to design a structured step-by-step plan prior to code generation. The LLM creates a comprehensive plan for code generation based on the problem description, which it then executes incrementally.

Authors' Contact Information: Ruwei Pan, Chongqing University, Chongqing, China, panruwei@stu.cqu.edu.cn; Hongyu Zhang, Chongqing University, Chongqing, China, hyzhang@cqu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA082
https://doi.org/10.1145/3832173

Recently, CodeCoT prompting [15] has integrated CoT with self-examination. The LLM improves the accuracy of the generated code by producing initial code and test cases, verifying code execution, and iteratively resolving errors as they are identified. Although existing prompting techniques have been proposed to guide LLMs, they still encounter several limitations due to their monolithic reasoning structure [2, 35]. This rigid structure limits LLMs' capacity to effectively break down complex programming problems into modular and independently solvable subproblems [43].

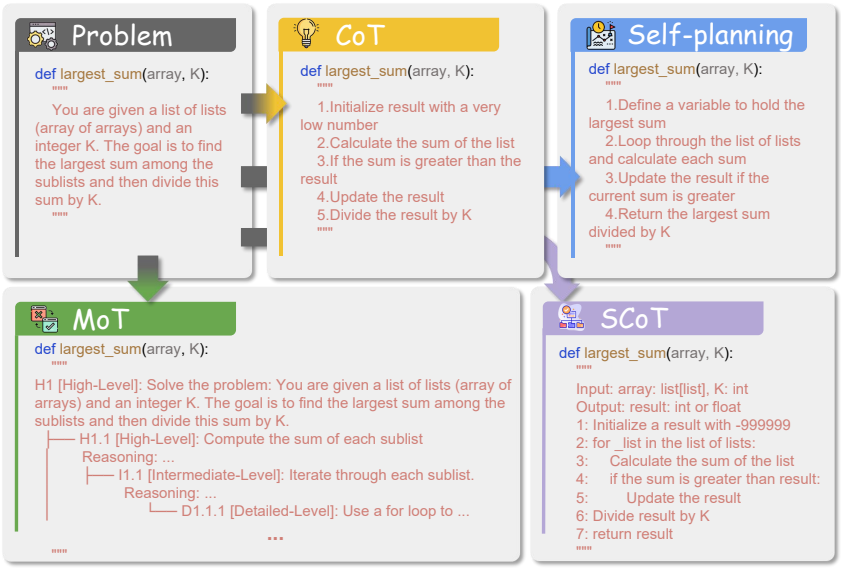


Fig. 1. Different Prompting Techniques

Analogous to the concept of monolithic programming in traditional software development, prompting techniques such as CoT suggest tasks step by step through a monolithic structure [42, 49]. Before 1970, software development primarily adhered to a linear, monolithic programming paradigm [37]. Although this monolithic approach was relatively effective at the time, its limitations gradually became evident as software size and complexity increased, leading to reduced development efficiency. To address the limitations of the monolithic approach, David Parnas introduced the concept of modularization in software development and highlighted the significance of hierarchical design [37–39]. He noted that by breaking complex problems into smaller, more manageable parts, developers can design and implement systems at a higher level of abstraction. Similarly, while explicit CoT methods have demonstrated effectiveness in specific tasks, their inherently linear structure limits their capacity to handle complex tasks. Researchers believe that the principles of modularization and hierarchical structure should also be incorporated into the reasoning processes of LLMs [11].

In this paper, we propose the MoT (**M**odularization **o**f **T**hought) prompting technique for code generation. MoT goes beyond traditional monolithic reasoning by introducing a structured approach that better captures the modular nature of programming problems. At its core, MoT constructs an MLR Graph (**M**ulti-**L**evel **R**easoning Graph), where each node represents a code task design, and each edge defines the logical dependencies between tasks. By providing an explicit structure of relationships and functionalities, the MLR Graph enhances LLMs' understanding of modular code components. Inspired by the principle of modularization in software development, MoT decomposes

complex programming problems into smaller, independent yet interdependent modules. These modules can be refined, adapted, and recombined, allowing for flexible composition of a coherent final solution. Through this modular and hierarchical approach, MoT enables LLMs to provide better solutions to programming problems.

We conduct extensive experiments to evaluate MoT with two state-of-the-art LLMs (i.e., GPT-4o-mini [34] and DeepSeek-R1 [10]) on eight benchmarks, including six function-level benchmarks (HumanEval [6], HumanEval-ET [9], HumanEval+ [29], MBPP [2], MBPP-ET [9], and MBPP+ [29]), one recent contest-style code generation benchmark (LiveCodeBench [16]), and one repository-level benchmark (RAL-Bench [36]). Our results show that MoT significantly outperforms all the compared prompting techniques with both LLMs on all benchmarks. For example, MoT achieves an average improvement of 0.03% to 32.85% in Pass@1 across all subjects compared to other techniques. The results show the effectiveness of MoT for improving code generation performance. Moreover, we construct two variants of MoT for the ablation study. The results confirm the contribution of the modularization principles and the MLR Graph prompting strategy to the overall MoT performance. We also conduct an experiment to analyze cost implications, demonstrating that MoT provides an effective approach to code generation.

We summarize our contributions in this work as follows:

- We propose a novel prompting technique, called MoT, to improve the code generation performance of LLMs by incorporating the modularization principles of software development into the reasoning process.
- We design a novel Multi-Level Reasoning Graph to enhance modular understanding and ensure better alignment between reasoning steps and the generated code.
- We conduct extensive experiments on two LLMs (i.e., GPT-4o-mini and DeepSeek-R1) with eight benchmarks, comparing them with six baselines to demonstrate the effectiveness of MoT in improving code generation performance.

2 Related Work

In recent years, LLMs have demonstrated significant potential in the field of code generation [17, 23, 24, 30, 31, 40]. Standard language models perform code completion and generation after autoregressive pre-training. Academia and industry have introduced various code LLMs, such as AlphaCode [27], CodeGen [32], CodeGeeX [48], InCoder [12], StarCoder [25], CodeLlama [41], and CodeT5+ [47]. Furthermore, general LLMs, such as ChatGPT [34] and DeepSeek [10], are also widely used for code generation. Building on these advances in model capability, recent studies have investigated how prompting techniques and structured reasoning can further improve code generation performance.

Prompting techniques have driven the advancement of LLM-based code generation [5]. Basic prompting techniques include zero-shot and few-shot prompting. Zero-shot prompting generates code based solely on the problem description without any examples, while few-shot prompting aids the LLM in understanding the input-output structure by incorporating examples [6]. Beyond these basic prompting techniques, representative prompting techniques include CoT [44], Self-Planning [18], and SCoT [22], which guide LLMs through explicit intermediate reasoning before code generation. CodeCoT [15] further combines reasoning with self-examination and iterative repair.

In addition to these prompting techniques, prior work has also explored structured reasoning methods beyond monolithic reasoning. Tree-of-Thoughts [45] and Graph-of-Thoughts [3] extend reasoning from a single linear chain to tree- or graph-structured exploration over multiple intermediate thoughts, typically involving exploration and selection among alternative reasoning paths.

More closely related to code generation, Parsel [46] decomposes problems into hierarchical function descriptions for compositional program synthesis, while CodeChain [20] and CodeTree [21] further improve code generation through iterative self-revision or tree-structured refinement under execution or agent feedback. These methods show the benefits of structured reasoning, but they differ from MoT in their primary mechanisms: ToT and GoT emphasize multi-path exploration, Parsel emphasizes compositional function specifications, and CodeChain and CodeTree emphasize iterative refinement or feedback-driven repair. In contrast, MoT focuses on the planning stage before final code generation by constructing a fixed-depth Multi-Level Reasoning graph as a lightweight planning scaffold to guide a single plan-to-code generation process. This design makes hierarchical decomposition directly usable for downstream code generation without introducing explicit search, backtracking, or iterative repair.

3 Approach

3.1 Overview

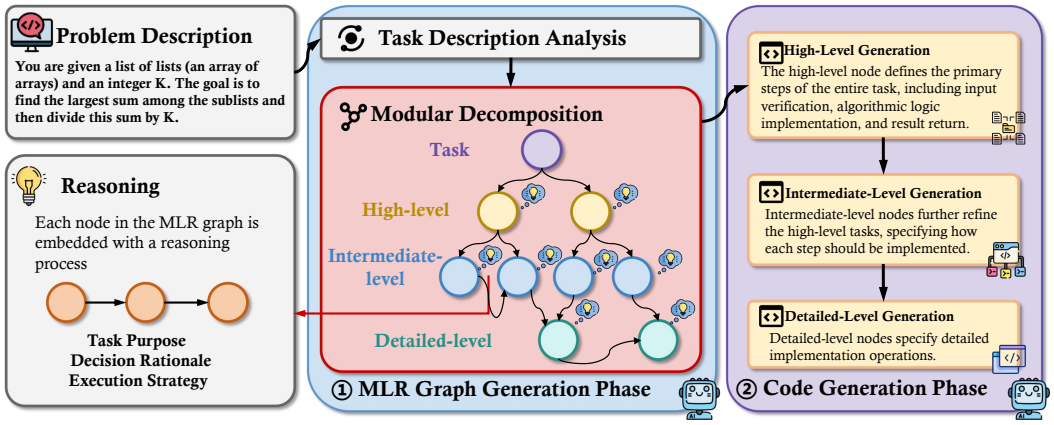


Fig. 2. The Overview of MoT.

We introduce MoT (**M**odularization-**o**f-**T**hought Prompting), a novel prompting technique based on MLR graphs (**M**ulti-**L**evel **R**easoning), to enhance LLMs' code generation. MoT applies modularization and hierarchical structures to guide logical reasoning and code generation using MLR graphs. Our core idea is to incorporate the modularization principles into the reasoning process. This novel Multi-Level Reasoning graph can improve modular understanding and ensure better alignment between reasoning steps and the generated code.

Figure 2 provides an overview of our method, which comprises two primary phases: (1) the MLR graph generation phase (see Section 3.2 for details), where the model analyzes the problem description and constructs an MLR graph of the programming problem; (2) the code generation phase (see Section 3.3 for details), where the model progressively generates code guided by the hierarchical MLR graph, thereby enhancing code accuracy through the modularization and hierarchical structure. The prompts that we used in our approach are shown in our repository.

3.2 MLR Graph Generation Phase

A Multi-Level Reasoning (MLR) graph is a structured representation that systematically structures reasoning steps into prompts, facilitating LLMs' comprehension of complex programming problems. In an MLR graph, each node indicates the rationale for its step by embedding the reasoning process.

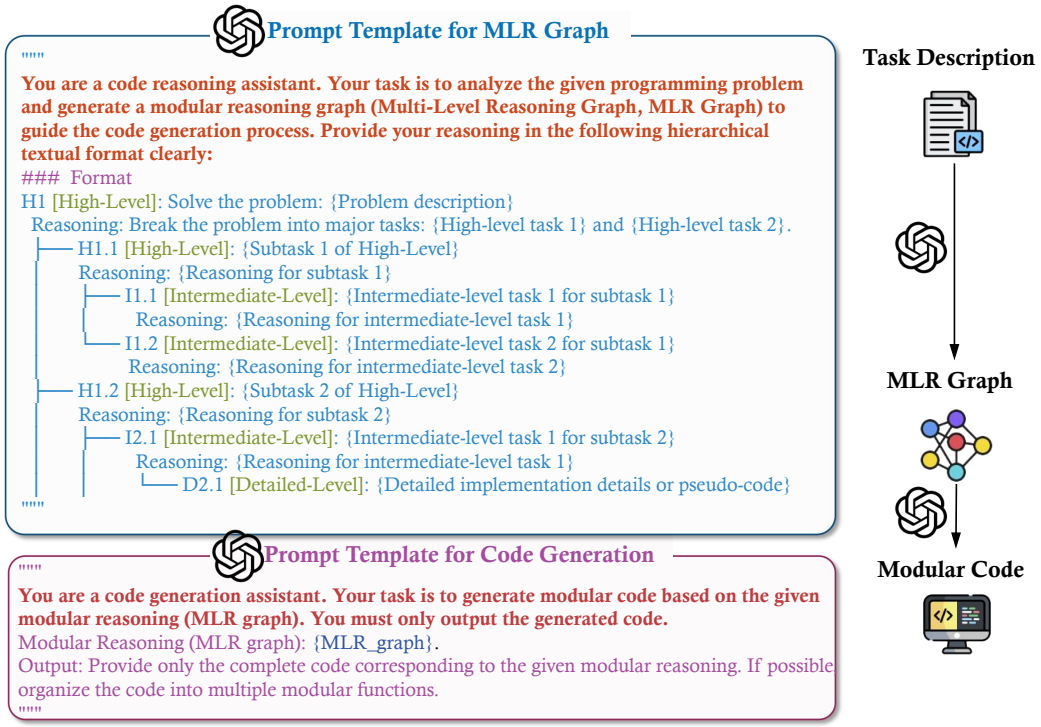


Fig. 3. Prompt Templates for the MLR Graph Generation and Modular Code Generation.

During the MLR graph generation phase, the LLM first generates an MLR graph, categorizing high-level, intermediate-level, and detailed-level designs, according to the description of programming problems. This structured prompting provides guidance during code generation, helps improve modular understanding, and ensures better alignment between reasoning steps and the generated code. Figure 3 illustrates the prompt template (in the blue area) utilized in the MLR graph generation phase.

3.3 Multi-Level Reasoning (MLR) Graph

In the MLR graph generation phase, MoT transforms task description analysis into an explicit hierarchical plan (the MLR graph) that guides subsequent code generation. The MLR graph uses a fixed three-level hierarchy to provide sufficient decomposition for complex tasks while preventing excessive fragmentation. Nodes represent sub-tasks at different abstraction levels, and directed edges capture decomposition-induced dependencies (a parent sub-task is refined into child sub-tasks). MLR graphs enable the model to provide modular designs for programming problems by decomposing tasks step by step and embedding a reasoning process.

Task description analysis: Firstly, the LLM extracts key task information from the programming problem, including the goal description, input and output specifications, and related constraints. The goal description typically outlines the core requirements of the programming problem. The LLM identifies relevant features based on the input specifications, such as data type, list length limits, or condition thresholds. The analysis results form the foundation for modular decomposition.

Modular decomposition: The MLR graph structures task design into three hierarchical levels:

- **High-Level:** The high-level node designs the primary steps of the entire task, including input verification, algorithmic logic implementation, and result return. Input validation ensures that the input data meets requirements, while the algorithmic logic implements the core steps of the task. The result return node is responsible for output generation and ensuring that the final result meets expectations.
- **Intermediate-Level:** Intermediate-level nodes further refine the high-level tasks, specifying how each step should be implemented. For example, input validation can be broken down into subtasks such as checking for empty lists and validating data types. Algorithm implementation can be decomposed into defining loop structures and making conditional judgments.
- **Detailed-Level:** Detailed-level nodes specify detailed implementation operations, such as calculating numerical differences or performing conditional checks. Detailed-level nodes describe local operations needed for correctness, and they are not intended to force a separate function for every step.

Each node in an MLR graph is embedded with a reasoning process, which explains the task purpose, decision rationale, and execution strategy:

- **Task Purpose:** Clarifies the need for the step, such as ensuring input validation to prevent errors in subsequent logic.
- **Decision Rationale:** Explains the reasoning behind specific choices, for example, using a nested loop to traverse all element combinations.
- **Execution Strategy:** Describes the operational details of the step, such as determining whether conditions are met or handling boundary cases.

By embedding the reasoning process, the LLMs can gain a better understanding of the design of each node in the MLR graph. In this way, the MLR graph not only delineates the logical structure of task decomposition but also systematically supplies the LLM with the necessary reasoning information for task execution, enhancing its code generation capabilities and logical consistency when processing complex tasks. For simpler tasks or nodes, the LLM can compress this reasoning process and inline the corresponding implementation to avoid over-modularized wrappers.

3.3.1 Shortcut Policy. MoT adopts a simple shortcut policy during MLR graph generation to control decomposition granularity and the verbosity of the reasoning process. This policy is training-free and is implemented as explicit prompting rules.

- **Stop expansion.** When generating the MLR graph level by level, the LLM may stop further expansion for a node if the sub-task is already specific enough to be implemented directly and does not benefit from additional planning. In particular, we stop expansion under the following conditions. ❶ The sub-task is directly implementable, for example, it can be expressed as a simple function. ❷ The sub-task does not require non-trivial control-flow design, such as multi-branch logic, loop structure decisions, or delicate boundary handling. Otherwise, the model continues expanding the sub-task to the next level (up to the fixed three-level hierarchy). This typically happens when the sub-task involves multiple constraints, requires non-trivial branching or looping, or contains boundary cases that benefit from explicit decomposition.
- **Reasoning compression.** Each node follows the same reasoning process schema: task purpose, decision rationale, and execution strategy. To reduce the risk of over-modularization, we apply the following rule. ❶ We use a compressed reasoning process when the sub-task is simple and self-evident. In this case, we keep a brief statement of the task purpose and execution strategy, and omit or shorten the decision rationale. ❷ We keep the full reasoning process when the sub-task involves non-trivial design choices or complex boundary handling.

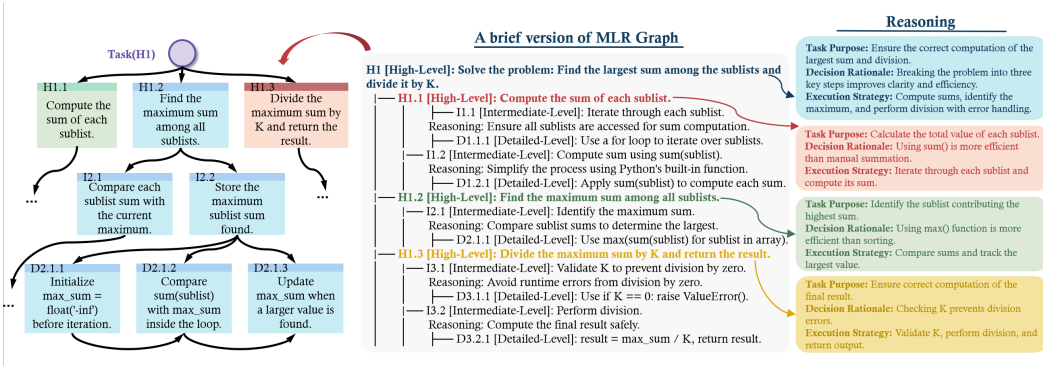


Fig. 4. Example of MLR Graph for the problem in Figure 2.

Typical cases include selecting control-flow structure, choosing data structures, coordinating multiple constraints, or addressing corner cases.

Therefore, MoT acts as a planning scaffold rather than a strict function-per-node design. Decomposition is used to improve planning quality, while simple nodes can be inlined and only reusable or non-trivial nodes are implemented as helper functions.

3.3.2 An Example. In Figure 4, we present a specific example of an MLR graph generated from the programming problem description in Figure 2 (partial). The complete MLR graph can be found in the repository. This figure demonstrates the modular decomposition and step-by-step reasoning process enabled by the MLR graph for complex programming tasks. The problem is to find the largest sum among the sublists and divide it by K. The MLR graph in Figure 4 has three high-level (e.g., "Compute the sum of each sublist"), five intermediate-level (e.g., "Iterate through each sublist"), and five detailed-level nodes (e.g., "Use a for loop to iterate over sublist"), enabling modular reasoning. For each node, the reasoning consists of three elements: Task Purpose, Decision Rationale, and Execution Strategy. For example, the task purpose of the high-level node "Compute the sum of each sublist" is to calculate the total value of each sublist for subsequent comparison. The decision rationale of the node emphasizes the use of the Python built-in function *sum()* for efficiency and simplicity. The execution strategy explicitly states that the calculation is performed by iterating over the sublist and invoking the *sum()* function.

3.4 Code Generation Phase

In the code generation phase, the LLM leverages the generated MLR graphs to progressively produce code according to the task decomposition structure defined by the nodes. The code generation phase builds upon the MLR graph established in the previous phase, enabling the model to effectively complete various components of complex tasks. In this phase, the previously generated MLR graph guides the LLM to generate structured and modular code. The model progressively produces the implementation by following the hierarchical nodes defined in the MLR graph (from high-level to detailed-level). Each node guides either a reusable helper function or a local code segment, depending on its complexity and reuse potential. Simple nodes can be inlined rather than materialized as separate functions.

3.4.1 An Example. To better illustrate how the MLR graph guides modular code generation, we present a generated final code based on the MLR graph in Figure 5. The code reflects the hierarchical and modular reasoning defined in the MLR graph.

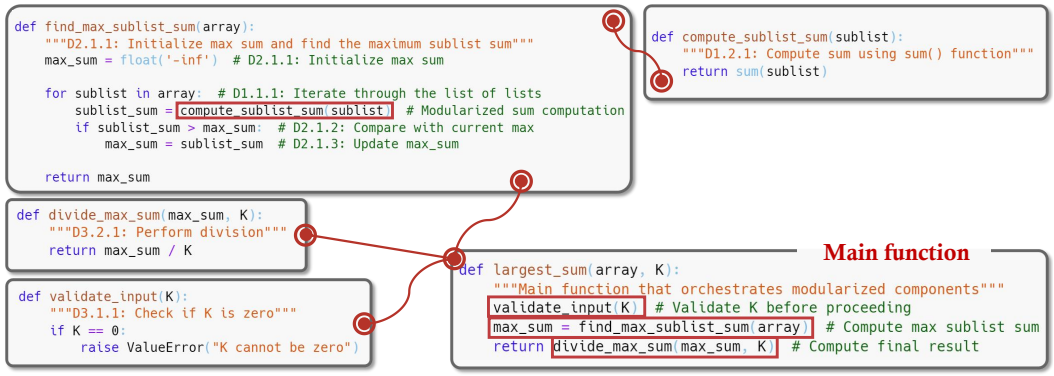


Fig. 5. Example of the generated code for the problem in Figure 2.

The high-level task is executed through *largest_sum()*, which serves as the primary computation function. It is responsible for calling each module sequentially to complete input verification, calculate the maximum sublist sum, and handle the final division operation. The intermediate task further refines the logic, where *find_max_sublist_sum()* is responsible for processing the input array and computing the maximum sublist sum, while *validate_input()* ensures the validity of K. The detailed task specifically implements the core computational steps, where *compute_sublist_sum()* efficiently calculates the sum of a single sublist using Python's built-in *sum()* function, and *divide_max_sum()* performs the final division operation, dividing the maximum sublist sum by K to obtain the final result. The entire execution process adheres to the hierarchical design of the MLR graph, ensuring modular reasoning and implementation. The structured guidance provided by the MLR graph can improve the comprehension of the programming problem and lead to better code.

3.5 Overall Algorithm

The MoT algorithm (Algorithm 1) constructs a hierarchical MLR graph to guide code generation, ensuring logical consistency and correctness. Although we use the term "MLR graph", in our prompt it is a hierarchical framework with only parent–child decomposition edges (i.e., a 3-level DAG without same-level links). This structure justifies the nested traversal in Algorithm 1. MoT consists of two phases.

In Phase I (MLR Graph Generation), the model parses the task description T and extracts key task elements, including goal G_T, I/O specification I_T, and constraints C_T. It then constructs the MLR graph level by level: High-level nodes N_T capture the main sub-tasks, Intermediate-level nodes N_M refine each high-level sub-task into implementable steps, and Detailed-level nodes N_B specify local operations and boundary handling. To reduce ambiguity, each node is associated with a reasoning process (task purpose, decision rationale, execution strategy). For simple nodes, we allow a compressed reasoning process to avoid over-modularization.

In Phase II (Code Generation), MoT first generates a code framework from the high-level nodes, including function names, signatures, and interfaces, which enforces alignment between plan nodes and code units. It then fills in the implementation by generating code snippets for detailed nodes and attaching them to the corresponding code unit. Finally, FinalizeCode integrates all code units into the complete output C_f.

Implementation note. Algorithm 1 expands the MLR graph in a depth-first manner (finish all children before moving to the next sibling). We choose depth-first expansion to preserve local

Algorithm 1 Modularization-of-Thought Prompting for Code Generation

Require: Task description T
Ensure: Final generated code C_f

Phase-I: MLR Graph Generation
$(G_T, I_T, C_T) \leftarrow$ ExtractTaskElements(T)
$MLRG \leftarrow$ InitializeMLR() # 3-level hierarchy with parent–child edges
$\{N_T\} \leftarrow$ GenerateHighLevel(G_T, I_T, C_T)
for all $N_T \in \{N_T\}$ **do**
  EmbedReasoning$(N_T, \texttt{compact})$
  $\{N_M\} \leftarrow$ GenerateIntermediateLevel(N_T)
  for all $N_M \in \{N_M\}$ **do**
    EmbedReasoning$(N_M, \texttt{compact})$
    $\{N_B\} \leftarrow$ GenerateDetailedLevel(N_M)
    for all $N_B \in \{N_B\}$ **do**
      EmbedReasoning$(N_B, \texttt{full})$
    end for
  end for
end for
$MLRG \leftarrow$ AssembleHierarchy$(\{N_T\}, \{N_M\}, \{N_B\})$ # Depth-first expansion for context consistency
Phase-II: Code Generation
$C \leftarrow$ InitializeCode()
$C \leftarrow$ GenerateScaffold$(MLRG, C)$ # names, signatures, interfaces, call relations
for all $N_T \in MLRG$ **do**
  for all $N_M \in N_T$ **do**
    for all $N_B \in N_M$ **do**
      $C \leftarrow$ GenerateCode(N_B, C) # attach to the mapped code unit; inline if trivial
    end for
  end for
end for
$C_f \leftarrow$ FinalizeCode(C) **return** C_f

context and reduce reasoning drift along a sub-task chain. The exact prompts for our method are included in our released artifact.

4 Evaluation

We evaluate MoT by answering the following research questions (RQs):

- **RQ1. How does MoT prompting perform in terms of accuracy compared to baselines?**
- **RQ2. How effective are the major components of MoT?**
- **RQ3. What are the cost implications of MoT?**
- **RQ4. What is the impact of MLR graph depth on code generation performance?**
- **RQ5. Is MoT robust to ambiguities and noisy variations in prompts?**

4.1 Datasets

To evaluate MoT, we have conducted extensive experiments on eight representative code generation datasets: HumanEval, HumanEval-ET, HumanEval+, MBPP, MBPP-ET, MBPP+, LiveCodeBench, and RAL-Bench.

- **HumanEval** [6] is a benchmark dataset designed to assess the code generation capabilities of large language models. It consists of 164 manually crafted Python programming problems, each accompanied by corresponding test cases to verify the correctness of the generated code.

- **MBPP** [2] is a dataset comprising diverse Python programming problems. It contains 974 code-generation tasks that cover a wide range of programming scenarios.
- **HumanEval-ET and MBPP-ET** [9] are extended versions of the original HumanEval and MBPP datasets.
- **HumanEval+ and MBPP+** [29] are enhanced versions of HumanEval and MBPP, with the EvalPlus framework augmenting each problem by adding a significantly larger set of test cases—approximately 80 times more than the original dataset.
- **LiveCodeBench** [16] is a continuously updated code generation benchmark collected from recent programming contests.
- **RAL-Bench** [36] is a repository-level generation benchmark. Given a natural language requirement, the model is required to generate a complete runnable repository.

4.2 Baselines

To evaluate MoT, we consider six typical prompting techniques for comparisons:

- **Zero-shot prompting** [6] is a method of generating code without utilizing any code examples. The model generates code based solely on problem descriptions.
- **Few-shot prompting** [6] allows LLMs to choose a few examples for understanding the relationship between problem and code. In our experiments, we adopt the 2-shot setting.
- **CoT prompting** [44] addresses complex code problems through step-by-step reasoning. When the model approaches a code problem, it first produces a sequence of intermediate steps, enhancing the logical coherence and correctness of the generated code.
- **Self-Planning prompting** [18] enables the model to formulate a step-by-step plan prior to code generation. The model creates a comprehensive plan for code generation based on the problem description, which it then executes incrementally.
- **SCoT prompting** [22] builds upon CoT prompting by leveraging program structures (i.e., sequence, branch, and loop structures) to generate intermediate reasoning steps.
- **CodeCoT prompting** [15] integrates CoT with self-examination mechanisms. The model improves the accuracy of generated code by producing logically coherent initial code and corresponding test cases, verifying code execution, and iteratively resolving errors as they are identified.

For CoT, SCoT, Self-Planning, and CodeCoT, to achieve fair comparisons, in our experiments, we adopt the prompts used in the respective papers.

4.3 Evaluation Metrics

Following existing work [9, 12, 18, 25, 27, 32, 41, 47, 48], we use executable test cases to verify the correctness of the generated code for each programming problem. Then, we employ **Pass@1** [6] and **AvgPassRatio** [13] metrics to assess the performance of LLMs in code generation.

Pass@1 measures the functional correctness of the generated code. This metric is used to evaluate the performance of generated code in test cases. Given a programming problem, an LLM generates one code instance. The problem is considered solved only if the instance passes all test cases. Pass@1 is the percentage of solved problems out of the total number of problems. The formula is as follows:

$$\text{Pass@1} := \mathbb{E}_{\text{Problems}}\left[1 - \frac{n-c}{n}\right] \tag{1}$$

AvgPassRatio measures the correctness of the generated code based on its performance across evaluation test cases. Pass@1 focuses on whether the generated code is completely correct in the test case, so we introduce AvgPassRatio to complement it. AvgPassRatio (APR) is calculated by

Table 1. Performance comparison across multiple datasets and models

Methods	HumanEval		HumanEval+		HumanEval-ET		MBPP		MBPP+	MBPP-ET	
	Pass@1	APR	Pass@1	APR	Pass@1	APR	Pass@1	APR	Pass@1	Pass@1	APR
GPT-4o-mini											
Zero-shot	88.4	86.2	81.1	30.8	87.1	75.0	59.9	59.7	47.9	53.6	52.6
Few-shot	82.3	63.6	76.2	30.7	81.7	64.7	49.1	49.9	40.6	48.4	42.7
CoT	87.8	79.2	82.9	29.4	87.8	86.5	61.2	62.4	48.6	54.1	53.5
Self-Planning	87.2	90.0	79.9	30.8	87.1	74.6	52.1	54.1	42.4	48.2	46.9
SCoT	86.6	93.1	78.7	30.8	86.0	74.8	63.9	63.9	51.4	55.6	57.2
CodeCoT	83.5	84.8	73.8	30.8	82.4	81.0	55.6	56.6	40.4	53.3	48.8
MoT	**92.1**	**96.9**	**83.5**	**30.9**	**91.5**	**88.0**	**73.9**	**79.7**	**58.1**	**58.9**	**64.4**
DeepSeek-R1											
Zero-shot	93.3	93.1	87.8	31.0	92.7	80.4	69.4	72.4	57.6	64.0	58.5
Few-shot	84.7	66.7	79.9	31.0	84.1	75.7	69.4	69.9	57.6	64.6	57.3
CoT	92.6	92.6	88.2	31.0	73.5	73.5	59.9	65.4	44.4	51.9	55.4
Self-Planning	85.4	88.3	79.3	30.8	85.3	72.3	68.4	69.2	55.4	65.5	56.4
SCoT	84.8	81.4	79.3	30.6	84.1	72.3	57.9	60.9	46.9	61.3	55.2
CodeCoT	66.5	64.9	60.4	31.3	65.9	58.3	69.2	88.7	56.6	64.5	74.5
MoT	**95.1**	**95.3**	**88.4**	**31.3**	**94.5**	**82.9**	**74.9**	**90.2**	**60.4**	**68.0**	**78.4**

determining the ratio of passed evaluation test cases to the total number of evaluation test cases for each problem and then averaging this ratio across all problems. Larger AvgPassRatio values indicate better code generation performance. Note that in our experiments, APR was not calculated for the MBPP+ dataset as it lacks complete test cases.

Furthermore, *benchmark-specific metrics* are used for LiveCodeBench and RAL-Bench following their original evaluation protocols [16, 36]. For LiveCodeBench, we report **Pass@1** on the full `release_v5` set. For RAL-Bench, we report **Fun.**, which is computed as the average pass rate of functional system tests across tasks.

4.4 Model Selection

To ensure practical and diverse model evaluation, we selected **GPT-4o-mini** and **DeepSeek-R1** (671B) because of their exceptional *Pass@1* performance and cost-efficiency. As shown in Table 2, these two models achieve top-tier accuracy on **HumanEval** and **HumanEval+** benchmarks while maintaining minimal API usage cost—especially GPT-4o-mini. Together, they represent two distinct paradigms: general-purpose LLMs and reasoning-specialized LLMs. Their favorable performance-to-cost ratio makes them ideal for scalable experimentation across a wide range of code generation tasks.

Table 2. Pass@1 score (%) and API cost comparison of models.

Model	HumanEval (Pass@1)	HumanEval+ (Pass@1)	Input Cost ($/1M)	Output Cost ($/1M)
DeepSeek-R1	**93.3**	**93.1**	–	–
GPT-4 Turbo	90.2	86.6	10.00	30.00
DeepSeek-V3	91.5	86.6	–	–
Claude 3.5 Sonnet	88.4	81.7	3.00	15.00
GPT-4o-mini	88.4	81.1	**0.15**	**0.60**

Table 3. Performance comparison on LiveCodeBench and RAL-Bench under different backbone LLMs.

Model	Method	LiveCodeBench	RAL-Bench
		Pass@1	Fun.
GPT-4o-mini	Zero-shot	26.7	12.2
	Few-shot	27.7	10.6
	CoT	33.6	13.5
	Self-Planning	21.9	15.4
	SCoT	27.9	16.1
	CodeCoT	33.8	16.4
	MoT	38.3	25.8
DeepSeek-R1	Zero-shot	64.8	16.2
	Few-shot	69.0	37.3
	CoT	74.2	36.0
	Self-Planning	66.4	40.8
	SCoT	64.7	44.0
	CodeCoT	67.3	45.1
	MoT	75.3	46.2

4.5 How Does MoT Prompting Perform in Terms of Accuracy Compared to Baselines? (RQ1)

1) Setup: To address RQ1, we apply MoT and six baseline techniques to GPT-4o-mini [34] and DeepSeek-R1 [10]. GPT-4o-mini is a compact, lightweight version of the GPT model, designed to balance response quality and processing speed. DeepSeek-R1 (671B) is an open-source LLM capable of performing high-precision NLP and code generation tasks. We use the default temperature parameters for GPT-4o-mini and DeepSeek-R1. Subsequently, we measure the effectiveness of each studied prompting technique on multiple benchmark settings. For function-level evaluation, we use six widely used benchmarks, including HumanEval, HumanEval-ET, HumanEval+, MBPP, MBPP-ET, and MBPP+, and report Pass@1 and AvgPassRatio metrics. To further reduce concerns about benchmark saturation, we also evaluate all studied prompting techniques on the full set of LiveCodeBench `release_v5`, which contains 1,055 recent programming problems, and report Pass@1 under the official evaluation protocol. In addition, we evaluate all studied prompting techniques on RAL-Bench to examine their effectiveness on repository-level generation tasks. To mitigate randomness, we run each experiment on the six function-level benchmarks 10 times and report the average results.

2) Results: In this section, we compare the performance of the MoT prompting technique across multiple datasets and models against several baseline methods. The results are provided in Table 1. The results provided **in the Pass@1 metric** show that MoT performs well across all datasets, achieving high scores on HumanEval (92.1), HumanEval+ (83.5), HumanEval-ET (91.5), and MBPP (73.9). In comparison, the performance of other prompting techniques is comparatively weaker. For instance, Self-Planning and SCoT do not achieve the performance level of MoT on multiple datasets, with particularly poor results on the MBPP+ and MBPP-ET datasets. These results demonstrate that MoT has distinct advantages in generating correct code. The results **in the APR metric** show that MoT achieves high APR scores across all datasets, particularly on HumanEval (96.9), HumanEval+ (30.9), and MBPP (79.7).

In addition to the six function-level benchmarks, we further evaluate MoT on LiveCodeBench and RAL-Bench. As shown in Table 3, MoT achieves the best results with both GPT-4o-mini and

DeepSeek-R1. On GPT-4o-mini, MoT reaches 38.3 Pass@1 on LiveCodeBench and 25.8 Fun. on RAL-Bench, outperforming the strongest baseline by 4.5 and 9.4 points, respectively. On DeepSeek-R1, MoT also obtains the best scores, with 75.3 Pass@1 on LiveCodeBench and 46.2 Fun. on RAL-Bench. These results suggest that MoT remains effective in both recent contest-style and repository-level code generation settings.

> ***Answer to RQ1:*** *MoT consistently improves performance across the eight benchmarks. These results show that MoT is effective across standard function-level, recent contest-style, and repository-level code generation tasks.*

4.6 How Effective Are the Major Components of MoT? (RQ2)

Table 4. Performance comparison with different MoT variants

Methods	HumanEval		HumanEval+		HumanEval-ET		MBPP		MBPP+	MBPP-ET	
	Pass@1	APR	Pass@1	APR	Pass@1	APR	Pass@1	APR	Pass@1	Pass@1	APR
GPT-4o-mini											
w/o MLR Graph	85.4	86.5	77.4	30.8	84.3	79.0	59.9	61.9	44.6	54.4	48.6
w/o Modularization	82.9	79.0	78.0	30.9	79.6	79.0	62.4	63.7	48.1	59.1	51.6
MoT	92.1	96.9	83.5	30.9	91.5	88.0	73.9	79.7	58.1	58.9	64.4
DeepSeek-R1											
w/o MLR Graph	64.6	87.5	59.1	30.1	61.5	57.9	36.3	43.5	31.3	36.0	32.3
w/o Modularization	58.5	82.4	56.1	31.0	56.8	57.7	41.6	43.1	35.3	40.5	33.1
MoT	95.1	95.3	88.4	31.3	94.5	82.9	74.9	90.2	60.4	68.0	78.4

1) Setup: To answer RQ2, we analyze how major components of MoT (as illustrated in Figure 2) affect its effectiveness. We compare the effectiveness of MoT with the following two variants:

- **w/o MLR Graph:** This variant removes the MLR Graph Generation (the graph structure within the red component in Figure 2), which delineates different hierarchical task levels (high-level, intermediate-level, and detailed-level tasks). It retains Modular Decomposition and Code Generation (the yellow component) to generate reasoning chains.
- **w/o Modularization:** This variant removes the Modular Decomposition component (the red highlighted area within component ① in Figure 2) and the modular Code Generation components (the yellow highlighted area within component ② in Figure 2). Unlike zero-shot or CoT prompting techniques, this variant still utilizes the MLR Graph's hierarchical reasoning structure (high-level, intermediate-level, and detailed-level). Instead of decomposing the task into modular subtasks for separate code generation, the LLM generates the complete final code in a single, monolithic step based on the hierarchical understanding.

The exact prompts for these variants are included in our released artifact.

2) Results: We evaluate MoT and its two variants (w/o MLR Graph and w/o Modularization) on GPT-4o-mini and DeepSeek-R1. As shown in Table 4, MoT consistently outperforms both variants across all datasets, confirming the importance of the MLR Graph and Modularization components. Notably, w/o Modularization still performs better than w/o MLR Graph on several datasets. This suggests that the MLR Graph provides the core planning scaffold, whereas modularized generation mainly affects how this scaffold is realized in code. In w/o MLR Graph, the model loses the explicit task decomposition and dependency structure between subproblems, while w/o Modularization still preserves the MLR graph but no longer generates code module by module according to the

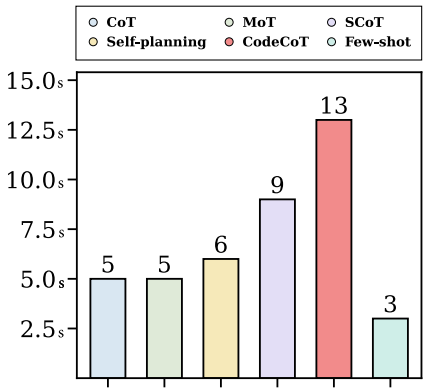
graph. Removing the MLR Graph leads to substantial drops in Pass@1 scores: from 92.1 to 85.4 on HumanEval (GPT-4o-mini), and from 95.1 to 64.6 on HumanEval (DeepSeek-R1), along with declines in APR, indicating the MLR Graph's key role in hierarchical task understanding. Without Modularization, the LLM adopts a monolithic reasoning approach, which also harms performance. On GPT-4o-mini, HumanEval drops from 92.1 to 82.9 and MBPP from 73.9 to 62.4. On DeepSeek-R1, MBPP drops sharply from 74.9 to 41.6, demonstrating the value of modular code generation. The complete MoT approach integrates both the MLR Graph and Modularization to achieve optimal performance across all tasks.

Answer to RQ2: *Our MoT prompting technique exhibits better performance than its variants, confirming the effectiveness and necessity of its major components.*

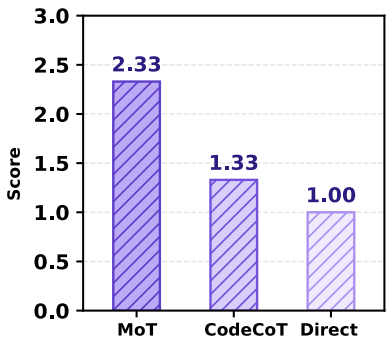
4.7 What Are the Cost Implications of MoT? (RQ3)

Table 5. Average per-task cost and token consumption on GPT-4o-mini.

Methods	HumanEval			MBPP		
	Cost ($)	In-token	Out-token	Cost ($)	In-token	Out-token
Zero-shot	0.0028	119.1	357.6	0.0025	45.1	340.5
Few-shot	0.0016	374.6	58.0	0.0022	578.8	157.0
CoT	0.0047	504.3	293.9	0.0044	510.4	390.5
Self-Planning	0.0062	521.6	364.1	0.0059	578.9	488.1
SCoT	0.0058	508.8	384.2	0.0052	559.7	479.6
CodeCoT	0.0071	535.5	428.7	0.0050	534.0	449.1
MoT	0.0044	501.3	282.4	0.0042	493.6	382.4



(a) Average generation time on HumanEval



(b) Evaluation on SWE-bench Lite

Fig. 6. Performance and efficiency comparison across different prompting techniques

1) Setup: To answer this RQ, we conducted cost evaluation experiments on 164 HumanEval and 399 MBPP problems. Although variants such as HumanEval-ET/+ and MBPP-ET/+ exist, they differ only in the number of test cases. For accurate cost measurement, each problem was solved individually, and the computational cost was calculated based on token usage under different prompting techniques.

2) Results: We compare the cost of MoT with those of typical prompting techniques, as shown in Table 5. Zero-shot and few-shot prompting techniques have notably lower computational costs compared to other prompting techniques. Specifically, zero-shot has the lowest In-token consumption, while few-shot achieves the lowest overall cost due to its lowest output-token consumption. Given the cost of an output token is typically much higher than that of an input token, few-shot has a lower cost than zero-shot. Although these two simple prompting techniques have low computational costs, they typically have inferior performance. The other results in Table 5 show that MoT demonstrates superior efficiency and lower computational cost compared to other typical prompting techniques.

Although MoT involves multiple generation steps, its intermediate outputs remain short and structured because the MLR graph follows a fixed three-level hierarchy and uses a predefined node-level reasoning schema. Moreover, the shortcut policy stops unnecessary expansion and compresses the reasoning for simple subtasks. Therefore, MoT can generate fewer output tokens than reasoning-oriented baselines such as CoT, Self-Planning, SCoT, and CodeCoT, which tend to generate longer free-form intermediate outputs.

As shown in Figure 6a, MoT remains efficient, with an average generation time of 5 seconds. Although few-shot is faster (3 seconds), it delivers much weaker generation performance. MoT is also faster than Self-Planning, SCoT, and CodeCoT, which take 6, 9, and 13 seconds, respectively. Unlike techniques such as CodeCoT, MoT does not require generating test cases, executing code, or iterative generation. Reducing unnecessary steps leads to a more efficient process, enabling MoT to achieve improved performance without compromising effectiveness.

> ***Answer to RQ3:*** *Based on these results, we conclude that MoT offers an efficient solution to achieve better performance, which incurs significantly lower cost than other prompting methods except simple zero-shot and few-shot.*

4.8 What Is the Impact of MLR Graph Depth on Code Generation Performance? (RQ4)

1) Setup: To analyse the effect of hierarchical depth in the MLR graph, we conduct an ablation study where we only vary the MLR graph depth and keep all other settings unchanged. We evaluate GPT-4o-mini on HumanEval and HumanEval+, measuring Pass@1 accuracy. To further examine whether the depth choice remains effective in a more realistic repository-level setting, we also evaluate different MLR graph depths on RAL-Bench using the same model.

2) Results: Results are shown in Table 6. The results show that performance peaks at 3 layers, suggesting that this configuration provides a practical balance between abstraction and granularity in the studied settings. Concretely, on HumanEval, Pass@1 rises from 83.5 (1 layer) to 85.4 (2 layers) and reaches 92.1 at 3 layers. On HumanEval+, the same trend holds, where Pass@1 increases from 75.6 to 79.9 and reaches the best result of 83.5 at 3 layers. On RAL-Bench, the 3-layer configuration also achieves the best functional score of 25.8, compared with 22.1 for 1 layer, 22.2 for 2 layers, and 23.9 for 4 layers. In contrast, adding an extra layer hurts performance. The 4-layer configuration drops to 87.2 on HumanEval and 78.0 on HumanEval+, which are 4.9 and 5.5 points lower than the 3-layer configuration, respectively. A similar trend is observed on RAL-Bench, where the 4-layer configuration drops by 1.9 points compared with the 3-layer configuration. These results indicate that shallower graphs (1–2 layers) may lack sufficient structure to guide reasoning, because they often stop at coarse steps and fail to make intermediate implementation decisions explicit. However, deeper graphs (4 layers) may introduce unnecessary redundancy and fragmentation, potentially over-decomposing the task into overly fine-grained nodes and increasing coordination overhead during modular generation. Overall, the depth study shows that the 3-layer hierarchy provides

Table 6. Impact of MLR graph depth on function-level and repository-level code generation.

Graph Depth	HumanEval		HumanEval+		RAL-Bench	
	Pass@1	Δ vs Avg (%)	Pass@1	Δ vs Avg (%)	Fun.	Δ vs Avg (%)
1 layer	83.5	−4.1	75.6	−4.6	22.1	−5.8
2 layers	85.4	−1.9	79.9	0.8	22.2	−5.7
3 layers	**92.1**	5.8	**83.5**	5.4	**25.8**	10.0
4 layers	87.2	0.2	78.0	−1.6	23.9	1.5
Average	87.1	0.0	79.3	0.0	23.5	0.0

Table 7. Comparative robustness under prompt perturbations on HumanEval using GPT-4o-mini. Delta values are computed as relative changes from the original-prompt setting to the perturbed-prompt setting. Smaller absolute changes indicate better robustness.

Method	Synonym			Shuffle			Typos		
	ΔPass@1	ΔBLEU	ΔEdit	ΔPass@1	ΔBLEU	ΔEdit	ΔPass@1	ΔBLEU	ΔEdit
CoT	-1.83%	**+0.12%**	-0.17%	-50.00%	**-1.01%**	+0.86%	+0.61%	**+0.10%**	-0.35%
Self-Planning	+5.49%	-0.51%	+0.81%	-47.56%	-4.22%	+7.08%	-0.61%	-0.55%	+1.08%
SCoT	+1.83%	-1.13%	+0.70%	-52.44%	-3.53%	+4.71%	-2.44%	-0.13%	+0.30%
CodeCoT	**-1.22%**	+0.33%	**+0.10%**	-54.88%	-2.84%	+3.59%	-4.27%	+0.11%	-0.92%
MoT	**-1.22%**	-0.55%	-0.17%	**-39.63%**	-2.35%	**+0.71%**	**-0.34%**	-0.12%	**-0.29%**

the best empirical trade-off across the evaluated function-level and repository-level settings. MoT adopts a default 3-layer hierarchy as a practical choice for balancing decomposition sufficiency and generation overhead.

Answer to RQ4: *Based on these results, we conclude that a 3-layer MLR hierarchy provides a strong practical default for balancing abstraction and granularity across the studied function-level and repository-level settings. Shallower hierarchies lack sufficient structured guidance, while deeper hierarchies introduce redundancy, fragmentation, and coordination overhead.* Therefore, MoT adopts 3 layers as the default depth for effective decomposition.

4.9 Is MoT Robust to Ambiguities and Noisy Variations in Prompts? (RQ5)

1) Setup: To study the robustness of different prompting methods to small variations or ambiguities in the problem description, we conducted a comparative evaluation on the HumanEval benchmark using GPT-4o-mini. We chose GPT-4o-mini because it is one of the main models in our study and provides a good balance between generation quality and evaluation cost for this comparative robustness analysis. We applied three types of controlled natural language perturbations to the input prompts:

- **Synonym substitution** (e.g., “calculate” → “compute”)
- **Word order shuffling** (within sentences)
- **Character-level typos** (e.g., “return” → “retrun”)

We then generate code for both original and perturbed prompts using MoT and four representative baselines, including CoT, Self-Planning, SCoT, and CodeCoT. We evaluate robustness from three aspects: Pass@1, BLEU, and normalized edit distance. For each metric m, we first compute its

value under the original-prompt setting, denoted as m_{orig}, and under the perturbed-prompt setting, denoted as m_{pert}. We then report the relative change as

$$\Delta m = \frac{m_{\text{pert}} - m_{\text{orig}}}{m_{\text{orig}}} \times 100\%.$$

Therefore, ΔPass@1 measures the relative task-level performance change under perturbation, ΔBLEU measures the relative change in BLEU score, and ΔEdit measures the relative change in normalized edit distance. Smaller absolute values indicate that the method is less affected by the perturbation. Table 7 summarizes the averaged comparative robustness results across all HumanEval tasks.

2) Results: As shown in Table 7, synonym substitution causes only small changes for most methods, suggesting that semantically equivalent wording changes are generally easy to handle. Word order shuffling is the most disruptive perturbation, but MoT shows the smallest Pass@1 drop (-39.63%) and the smallest absolute ΔEdit among all methods. Under character-level typos, MoT also shows the smallest Pass@1 change (-0.34%) and a small edit-distance change (-0.29%). Overall, MoT is more stable than representative prompting baselines under the more disruptive perturbations.

> ***Answer to RQ5:*** *MoT shows competitive robustness under noisy and ambiguous prompt perturbations. It achieves smaller performance changes in several settings, while word-order shuffling remains a challenging case.*

5 Discussion

5.1 How Does MoT Perform on Other Real-World Software Development Tasks?

The RQ1 evaluation has already included LiveCodeBench and RAL-Bench to assess MoT on recent code generation tasks and repository-level generation tasks. In this section, we further examine MoT on real-world repository bug fixing. To this end, we evaluate MoT on the official **SWE-bench Lite** set, which contains 300 real-world software issue tasks, using GPT-4o-mini. This experiment provides preliminary evidence on whether MoT can help with repository-level bug fixing. For each task, all methods receive the same issue description and the same repository context, and are evaluated under the same patch-generation and test-execution protocol. This setting keeps the comparison focused on the prompting strategy under a shared input context. We compare MoT with two representative baselines. *Direct* serves as a one-pass generation baseline without explicit decomposition or structured reasoning. *CodeCoT* is selected because it is one of the strongest prompting baselines in RQ1 and incorporates self-examination during code generation. This comparison allows us to examine whether MoT brings benefits beyond direct generation and reasoning-based prompting.

Figure 6b compares Direct, CodeCoT, and MoT on SWE-bench Lite. Overall, MoT achieves competitive performance. These results suggest that MoT can be applied to repository editing tasks beyond standalone code generation. More comprehensive evaluation on real-world software engineering benchmarks, especially with stronger retrieval, localization, and feedback-driven repair mechanisms, remains future work.

5.2 How Does MoT Compare with Agent-Based Methods?

To complement the comparisons with prompting-only baselines in RQ1, we further compare MoT with representative agent-based repository-level code generation methods. We present this comparison separately from RQ1 because agent-based methods differ from prompting-only baselines in both inference procedure and computational cost. While RQ1 focuses on prompting baselines

Table 8. Comparison with agent-based baselines on repository-level generation benchmarks.

Method	NL2Repo-Bench (GPT-5)				RAL-Bench (GPT-5)
	Overall	Easy	Medium	Hard	Fun.
Direct	21.7	38.4	20.7	9.6	38.5
SE-Agent	25.4	44.5	23.6	12.6	44.7
AlphaEvolve	25.1	41.8	25.1	11.7	43.7
CSE	19.7	41.1	15.8	7.9	34.3
MoT	**29.7**	34.8	**36.8**	**20.7**	**46.2**

evaluated under the same experimental protocol, agent-based methods often involve multi-step coordination, search, or iterative refinement [28]. This comparison helps clarify how MoT performs relative to agent-based repository-level methods.

We evaluate MoT on RAL-Bench [36] and NL2Repo-Bench [8] for this separate comparison. RAL-Bench is also used in the main evaluation, while NL2Repo-Bench is included here because its difficulty-stratified repository-level tasks are suitable for comparing MoT with agent-based methods. Together, these two benchmarks provide complementary views of repository-level generation. For NL2Repo-Bench, which is used only in this separate discussion comparison, we report **Overall**, **Easy**, **Medium**, and **Hard** scores following the original benchmark protocol [8]. Each score is computed as the average pytest pass rate of generated repositories. We compare MoT with four representative baselines. *Direct* is a simple one-pass generation baseline without explicit decomposition. *SE-Agent* is a software-engineering-oriented agent baseline for repository-level tasks. *AlphaEvolve* [33] and *CSE* [14] are repository-level baselines that move beyond simple direct prompting. We use GPT-5 as the shared backbone for all methods so that the comparison is conducted under the same model setting and focuses on the generation strategy of each method.

As shown in Table 8, MoT achieves the best results on both benchmarks. Its advantage is especially clear on the medium and hard subsets of NL2Repo-Bench. It also obtains the highest functional score on RAL-Bench. A likely reason is that MoT introduces a generation-aware hierarchical plan that directly connects task decomposition with downstream code generation, while avoiding the extra coordination or search complexity of agent-based methods. These results suggest that MoT is effective for repository-level code generation tasks.

5.3 How Does MoT Compare with Structured Reasoning Methods?

We present this comparison separately from RQ1 because these methods differ markedly from standard prompting baselines in both inference procedure and cost. While RQ1 focuses on standard prompting baselines, this subsection examines stronger structured reasoning methods, including ToT, GoT, Parsel, CodeChain, and CodeTree. We evaluate these methods on HumanEval and RAL-Bench, which provide complementary views of code generation performance. HumanEval measures function-level code generation under a standard benchmark setting, while RAL-Bench evaluates functional effectiveness on more realistic repository-level tasks. We use GPT-4o-mini as the shared backbone. All compared methods are reproduced based on their original papers or released implementations. We therefore report both effectiveness and full-benchmark cost to provide a more faithful comparison.

As shown in Table 9, MoT remains competitive with closely related structured reasoning methods on both HumanEval and RAL-Bench, while also maintaining favorable efficiency. This suggests that MoT can achieve strong performance without relying on the heavier search or refinement procedures used by some alternative methods.

Table 9. Performance and full-benchmark cost comparison with structured reasoning and refinement methods on HumanEval and RAL-Bench using GPT-4o-mini.

Method	Performance		HumanEval Cost		RAL-Bench Cost	
	HumanEval Pass@1	RAL-Bench Fun.	Token	Time (s)	Token	Time (s)
ToT	84.1	8.5	1817.3	15.58	4949.0	36.81
GoT	90.2	16.6	2712.3	20.04	4546.1	34.99
Parsel	71.8	14.0	2784.7	19.95	5489.4	32.14
CodeChain	90.9	15.3	4323.0	25.43	8389.9	45.93
CodeTree	84.8	13.3	3503.6	21.18	9722.1	52.06
MoT	**92.1**	**25.8**	783.7	5.12	3431.6	31.69

5.4 Does MoT Lead to Over-Modularization?

Over-modularization is the core risk. A common concern of modular prompting is over-modularization, where the generated solution introduces unnecessary functions or modules. From a software engineering perspective, such over-modularization can increase complexity, reduce readability, and add avoidable indirection. This risk is most pronounced if modular reasoning units are naively mapped to isolated functions.

MoT reduces over-modularization risk. MoT mitigates this risk in two ways. First, it fixes decomposition to three levels, which bounds granularity and prevents unbounded splitting into micro-tasks. Second, MoT does not require each node to become a separate function: reusable nodes can be implemented as helpers, while simple nodes can be inlined. For simple nodes, we also keep the reasoning concise to avoid pushing the model toward over-modularization. This aligns with our depth study: decomposition helps up to a point, whereas over-modularization can introduce redundancy, coordination overhead, and unnecessary function-call overhead. Overall, MoT should be viewed as a planning framework rather than a strict rule for function boundaries. MoT is not designed as a backtracking-based repair framework, but as a planning scaffold for improving first-pass generation. Still, the explicit MLR graph improves traceability by exposing node-level decomposition and reasoning, which can make failures easier to localize than in a monolithic generation process. In this sense, MoT is complementary to feedback-driven repair methods rather than a replacement for them. Future work could make the modularization more adaptive by deciding more directly when to inline code and when to create a helper function.

6 Threats to Validity

The first potential threat relates to the stochasticity of LLMs, which could affect the measured accuracy and stability of the results. To mitigate this threat, we repeat experiments across benchmarks and report the averaged results. This reduces the influence of random fluctuations and improves the robustness of our empirical observations.

The second potential threat concerns the generalizability of MoT across programming languages. Our current evaluation primarily focuses on Python, while programming languages differ in syntax, abstraction mechanisms, and library ecosystems, which may affect the effectiveness of MoT. We choose Python because it is widely used and representative in code-generation evaluation. In future work, we plan to further validate MoT on a broader range of programming languages.

The third potential threat concerns the generalizability of our evaluation to broader software engineering tasks. Evidence from function-level benchmarks and repository-level generation alone may be insufficient to support generality across diverse maintenance scenarios. To mitigate this threat, we extend our evaluation to repository-level generation settings, including RAL-Bench and

a separate comparison on NL2Repo-Bench, and include a preliminary SWE-bench Lite study on repository-level bug fixing. However, our current maintenance-oriented evaluation is still limited to bug fixing and does not fully cover other tasks such as feature enhancement. More comprehensive evaluation on broader software engineering tasks, such as FEA-Bench [26], remains future work.

The fourth potential threat concerns the suitability of the fixed three-level MLR hierarchy. The most appropriate hierarchy depth may vary across tasks and settings. To mitigate this threat, we conduct a dedicated depth analysis and compare multiple hierarchy depths in the current evaluation setting. Our results show that a three-level hierarchy works well for the studied tasks. Exploring the optimal hierarchy depth across diverse settings remains future work.

7 Conclusion

In this paper, we propose a novel prompting technique MoT to improve the code generation performance of LLMs. Different from the existing prompting techniques, MoT incorporates the modularization principles into the reasoning process. It introduces a novel Multi-Level Reasoning (MLR) graph to enhance modular understanding and ensure alignment between reasoning and the generated code by hierarchically organizing reasoning steps with its modular structure. We have conducted an extensive study with two advanced LLMs on eight benchmarks. We further provide additional analyses on NL2Repo-Bench and SWE-bench Lite. The results confirm the superiority of MoT over state-of-the-art prompting techniques. In future work, we will extend MoT to multi-language and real-world software tasks, and explore adaptive decomposition strategies that better balance structure and over-modularization.

8 Data Availability

To ensure the reproducibility of our results and to provide transparency in our research, we have made all related scripts and data publicly available. All resources (our source code, experimental data, and concrete examples of prompts and generated code) can be accessed as part of our artifact, which is available at **https://github.com/Wwstarry/Modularization_of_Thought**.

References

[1] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2022. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868* (2022).

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[3] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 17682–17690.

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[5] Kaiyan Chang, Songcheng Xu, Chenglong Wang, Yingfeng Luo, Xiaoqian Liu, Tong Xiao, and Jingbo Zhu. 2024. Efficient prompting methods for large language models: A survey. *arXiv preprint arXiv:2404.01077* (2024).

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[7] Le Deng, Zhonghao Jiang, Jialun Cao, Michael Pradel, and Zhongxin Liu. 2025. Nocode-bench: A benchmark for evaluating natural language-driven feature addition. *arXiv preprint arXiv:2507.18130* (2025).

[8] Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, et al. 2025. NL2Repo-Bench: Towards Long-Horizon Repository Generation Evaluation of Coding Agents. *arXiv preprint arXiv:2512.12730* (2025).

[9] Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2023. Codescore: Evaluating code generation by learning code execution. *arXiv preprint arXiv:2301.09043* (2023).

[10] DeepSeek-AI et al. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. arXiv:2401.02954 [cs.CL] https://arxiv.org/abs/2401.02954

[11] Martin Fowler. 2018. *Refactoring: improving the design of existing code.* Addison-Wesley Professional.

[12] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).

[13] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938* (2021).

[14] Tu Hu, Ronghao Chen, Shuo Zhang, Jianghao Yin, Mou Xiao Feng, Jingping Liu, Shaolei Zhang, Wenqi Jiang, Yuqi Fang, Sen Hu, et al. 2026. Controlled self-evolution for algorithmic code optimization. *arXiv preprint arXiv:2601.07348* (2026).

[15] Dong Huang, Qingwen Bu, and Heming Cui. 2023. Codecot and beyond: Learning to program and test like a developer. *arXiv preprint arXiv:2308.08784* (2023).

[16] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974* (2024).

[17] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: Large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering*. 1219–1231.

[18] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.

[19] Zhonghao Jiang, David Lo, and Zhongxin Liu. 2025. Agentic Software Issue Resolution with Large Language Models: A Survey. *arXiv preprint arXiv:2512.22256* (2025).

[20] Hung Le, Hailin Chen, Amrita Saha, Akash Gokul, Doyen Sahoo, and Shafiq Joty. 2023. Codechain: Towards modular code generation through chain of self-revisions with representative sub-modules. *arXiv preprint arXiv:2310.08992* (2023).

[21] Jierui Li, Hung Le, Yingbo Zhou, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2025. Codetree: Agent-guided tree search for code generation with large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 3711–3726.

[22] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23.

[23] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. Skcoder: A sketch-based approach for automatic code generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2124–2135.

[24] Jia Allen Li, Yongmin Li, Ge Li, Xing Hu, Xin Xia, and Zhi Jin. 2021. Editsum: A retrieve-and-edit framework for source code summarization. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 155–166.

[25] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[26] Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. 2025. Fea-bench: A benchmark for evaluating repository-level code generation for feature implementation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 17160–17176.

[27] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

[28] Jiaye Lin, Yifu Guo, Yuzhen Han, Sen Hu, Ziyi Ni, Licheng Wang, Mingguang Chen, Hongzhang Liu, Ronghao Chen, Yangfan He, et al. 2025. Se-agent: Self-evolution trajectory optimization in multi-step reasoning with llm-based agents. *arXiv preprint arXiv:2508.02085* (2025).

[29] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Thirty-seventh Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=1qvx610Cu7

[30] Shuai Lu, Nan Duan, Hojae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. 2022. Reacc: A retrieval-augmented code completion framework. *arXiv preprint arXiv:2203.07722* (2022).
[31] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568* (2023).
[32] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).
[33] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. 2025. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131* (2025).
[34] OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774
[35] Ruwei Pan, Hongyu Zhang, and Chao Liu. 2025. CodeCoR: An LLM-Based Self-Reflective Multi-Agent Framework for Code Generation. *arXiv preprint arXiv:2501.07811* (2025).
[36] Ruwei Pan, Yakun Zhang, Qingyuan Liang, Yueheng Zhu, Chao Liu, Lu Zhang, and Hongyu Zhang. 2026. RAL-Bench: Benchmarking for Application-Level Functional Correctness and Non-Functional Quality Attributes. *arXiv preprint arXiv:2602.03462* (2026).
[37] David Lorge Parnas. 1972. On the criteria to be used in decomposing systems into modules. *Commun. ACM* 15, 12 (1972), 1053–1058.
[38] David L Parnas. 1978. On a "buzzword": Hierarchical structure. *Programming Methodology: A Collection of Articles by Members of IFIP WG2. 3* (1978), 335–342.
[39] David L Parnas. 2002. The secret history of information hiding. *Software pioneers: contributions to software engineering* (2002), 398–409.
[40] Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. *arXiv preprint arXiv:2108.11601* (2021).
[41] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] https://arxiv.org/abs/2308.12950
[42] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).
[43] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* (2022).
[44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
[45] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
[46] Eric Zelikman, Qian Huang, Gabriel Poesia, Noah Goodman, and Nick Haber. 2023. Parsel: Algorithmic Reasoning with Language Models by Composing Decompositions. *Advances in Neural Information Processing Systems* 36 (2023), 31466–31523.
[47] Jiaming Zheng, Peng Xu, Ji Liu, Xuanjing Huang, and Xipeng Qiu. 2023. CodeT5+: Open Code Generation Model Pretrained on Text-to-Code and Code-to-Code Tasks. *arXiv preprint arXiv:2302.07930* (2023).
[48] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x. *arXiv preprint arXiv:2303.17568* (2023).
[49] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625* (2022).

Received 2026-01-29; accepted 2026-06-25